

APOSTILA GITHUB — IMPULSO DIGITAL

Para: Eduardo Barros · operador Impulso Digital **Nível assumido:** mediano (sabe commit/push/branch básico via Cursor)
Objetivo: sair de "uso porque o Cursor me obriga" pra "GitHub é minha vitrine técnica e meu sistema de versionamento sério" **Tempo de leitura:** ~45 min · **Tempo pra aplicar tudo:** ~3h em uma tarde

ÍNDICE

- 1. [O que é Git × O que é GitHub](#)
- 2. [Anatomia de um repositório](#)
- 3. [Anatomia do seu perfil](#)
- 4. [Fluxo do dia-a-dia no Cursor/Code](#)
- 5. [.gitignore e arquivos sensíveis](#)
- 6. [Branches — trabalhar sem quebrar nada](#)
- 7. [Profissionalizar seu perfil](#)
- 8. [Profissionalizar seus repositórios](#)
- 9. [Colaboração — receber ajuda do Lucas \(ou contratado\)](#)
- 10. [Segurança — não vazar senha/chave](#)
- 11. [Erros comuns e como sair deles](#)
- 12. [Cheatsheet de comandos](#)
- 13. [Glossário PT-EN](#)
- 14. [Checklist — "deixar profissional em 3 horas"](#)

1. GIT × GITHUB

Git = programa que roda no teu computador. Controla versões de arquivos. É offline, não tem nada a ver com internet.
GitHub = site/serviço da Microsoft que hospeda repositórios Git na nuvem. É a "rede social" do Git.

Analogia simples:

- Git = Word + função "salvar versão histórica"
- GitHub = Google Drive onde tu sobe esses arquivos pra compartilhar

Tu pode usar Git sem GitHub (versionar local). Tu não pode usar GitHub sem Git.

Outros como GitHub: GitLab, Bitbucket, Codeberg. GitHub é o dominante (~100M devs).

2. ANATOMIA DE UM REPOSITÓRIO

Um **repositório** (repo) é uma pasta de projeto com histórico Git ativado.

Quando tu abre `github.com/ImpulsoDigital063/impulso-digital-nextjs`, vê isso:

Code	Issues	Pull Requests	Actions	Projects	Settings
main ▾ 1,234 commits ⓘ 5 branches 3 tags					
src/		"feat: novo header"		2h ago	
public/		"fix: logo size"		1d ago	
README.md		"docs: como rodar"		3d ago	
package.json		"chore: bump next"		1w ago	
.gitignore		"init"		2m ago	

← abas

← branch + estatísticas

Abas que importam

Aba	Pra que serve	Quando tu usa
Code	ver os arquivos	sempre
Issues	reportar bugs / pedir features	quando algo quebra ou tu quer planejar
Pull Requests	propor mudanças (PR)	colaboração com outros
Actions	rodar automação (CI/CD)	testes automáticos, deploy
Settings	configurar repo (visibilidade, colaboradores, branches)	uma vez por repo

Conceitos centrais

Termo	O que é
Commit	uma "foto" do projeto num momento. Tem hash (ex: <code>a3f8c2</code>), autor, data, mensagem
Branch	linha do tempo paralela. <code>main</code> é a principal. <code>feature/login</code> é uma alternativa
HEAD	"onde tu tá agora". Aponta pra um commit específico em uma branch
Remote	versão do repo num servidor remoto (GitHub). Apelido padrão: <code>origin</code>
Origin	o remote principal (geralmente teu fork no GitHub)
Upstream	o repo original (quando tu fez fork)
Push	enviar commits locais pro remote
Pull	trazer commits do remote pro local
Fetch	trazer informação do remote SEM mesclar (só atualiza referências)
Merge	juntar duas branches
Rebase	"re-aplicar" commits em cima de outra base (alternativa ao merge)
Tag	marcador permanente num commit (geralmente versão: <code>v1.0.0</code>)
Release	tag + notas + artefatos publicados na aba Releases

3. ANATOMIA DO SEU PERFIL

Teu perfil: `github.com/ImpulsoDigital063`

[foto] Impulso Digital
ImpulsoDigital063
[Edit profile]

Overview Repositories(14) Projects Packages Stars

Popular repositories

repo1 repo2

...

824 contributions in the last year [gráfico verde]

Contribution activity
May 2026 ▶ Created 293 commits in 7 repositories

← abas do perfil
← os 6 "pinned"

Componentes do perfil que tu controla

- Foto** — Settings → Profile → Profile picture. Use logo da Impulso.
- Nome** — Settings → Profile → Name. Use "Impulso Digital".
- Bio** — 160 chars. Ex: "Agência de tráfego e tecnologia. SaaS, LPs, automação."
- Localização / website / email / Twitter** — opcional, mas link pro site da Impulso ajuda.
- Pinned repositories** — 6 cards que aparecem na home. Clica em "Customize your pins". Os 6 que aparecem hoje saíram automático (mais estrelas/forks).
- Profile README** — texto rico no topo da home. Explicado em [\\$7](#).

4. FLUXO CURSOR/CODE → GITHUB

Cenário comum: tu acabou um projeto no Cursor, quer subir pro GitHub.

Cenário A — Projeto novo, criando do zero

Passo 1 — Criar o repo no GitHub (pelo site)

- Abre `github.com/new`
- Owner: `ImpulsoDigital063`
- Repository name: `nome-do-projeto` (kebab-case)
- Description: 1 linha curta ("LP de captação para clínica X")
- Public / Private: **Private** se for projeto de cliente pago; Public se for portfólio/open-source
- **NÃO marca** "Add a README" / "Add .gitignore" / "Choose a license" — tu vai colocar isso do projeto local
- Create repository

GitHub te dá uma URL: `https://github.com/ImpulsoDigital063/nome-do-projeto.git`

Passo 2 — No Cursor (terminal integrado, Ctrl+`)

```
cd C:\Users\Usuario\projeto-x
git init
git add .
git commit -m "feat: inicial"
git branch -M main
git remote add origin https://github.com/ImpulsoDigital063/projeto-x.git
git push -u origin main
```

inicia versionamento local
marca todos os arquivos
primeira foto
renomeia branch pra main
primeiro push

Pronto. Atualiza a página do GitHub e tá tudo lá.

O `-u` no push = "lembra que essa branch local rastreia essa remota". Depois disso tu só faz `git push` sem mais nada.

Cenário B — Projeto que tu já mexeu no Cursor sem Git

Mesma coisa. O `git init` funciona em pasta com arquivos existentes.

Cenário C — Trabalho diário (depois que repo já tá no GitHub)

```
# editar arquivos no Cursor
git status
git add .
git commit -m "fix: erro no header"
git push
```

vê o que mudou
marca tudo
foto
envia

Ou pela interface do Cursor:

- Aba "Source Control" (Ctrl+Shift+G)
- Vê arquivos modificados (M), novos (U), deletados (D)
- "+" pra marcar (stage)
- Escreve mensagem no topo
- Ctrl+Enter pra commitar
- "..." → Push

Padrão de mensagem de commit (Conventional Commits)

Adota isso. Vira reflexo em 1 semana e teu histórico fica legível:

```
<tipo>(<escopo>): <descrição curta minúscula>

feat(header): adiciona menu mobile
fix(form): valida email antes de enviar
docs(readme): adiciona seção de stack
chore(deps): atualiza next pra 15.2
refactor(api): extrai cliente do supabase
style(css): ajusta espaçamento do hero
test(login): adiciona teste de redirect
ops(db): adiciona migration v62
```

Tipos comuns: `feat`, `fix`, `docs`, `chore`, `refactor`, `style`, `test`, `ops`, `perf`.

Por que importa: ferramenta `release-please` ou `standard-version` lê isso e gera CHANGELOG sozinha. Lucas Passos usa esse padrão no GitHub dele — é sinal de maturidade.

Cenário D — Clonar repo existente (teu ou de outro)

```
git clone https://github.com/ImpulsoDigital063/projeto-x.git
cd projeto-x
npm install
```

`git clone` traz repo completo + histórico + remote já configurado.

5. .GITIGNORE

Arquivo na raiz do projeto que diz pro Git "ignora esses caminhos, não versiona".

O que SEMPRE entra no .gitignore (Next.js / Node):

```
# dependências (são pesadas e instaláveis via package.json)
node_modules/

# build
.next/
out/
dist/
build/

# environment (TEM SENHAS)
.env
.env.local
.env*.local

# logs
*.log
npm-debug.log*
yarn-debug.log*

# OS
.DS_Store
Thumbs.db

# editor
.vscode/
.idea/
*.swp

# cache / temp
.cache/
.turbo/
.vercel/
```

Por que isso é crítico: se tu commitar `.env` com `SUPABASE_SERVICE_ROLE_KEY=eyJ ...`, qualquer pessoa no mundo lê tua chave. E o histórico Git não esquece — não basta deletar depois, tem que **revogar a chave no Supabase**.

Como aplicar em projeto existente:

```
# se já commitou node_modules ou .env por engano:
git rm -r --cached node_modules
git rm --cached .env
# depois commita o .gitignore atualizado
git add .gitignore
git commit -m "chore: gitignore node_modules e env"
git push
```

Truque: `npx gitignore node` gera um `.gitignore` completo pra Node. Tem versão pra `next`, `python`, etc.

6. BRANCHES

Pensa em branch como uma linha do tempo paralela. Tu trabalha numa branch separada pra não quebrar a `main` (que normalmente é o código em produção).

Fluxo padrão

```
main:  o-o-o-o-----o  ← código estável, em produção
        \               /
feature/x:  o-o-o-o-o-o-o  ← tu trabalhas aqui
```

```
git checkout -b feature/menu-mobile # cria branch e troca
# ...edita, commita várias vezes...
git push -u origin feature/menu-mobile

# depois abre Pull Request no GitHub pra mergear na main
```

Comandos de branch

```
git branch          # lista branches locais
git branch -a       # inclui remotas
git checkout main    # troca pra main
git checkout -b feature/x # cria + troca
git branch -d feature/x # deleta (só se já mergeada)
git push origin --delete feature/x # deleta no remote
```

Quando tu precisas de branch (e quando não)

Não precisa se tu trabalhas sozinho num projeto pequeno (LP de cliente) e o que tu commitas já é o que vai pra produção. `main` direto serve.

Precisa quando:

- Vai mexer em algo arriscado e quer poder abandonar
- Vai colaborar com Lucas/outro dev (cada um na sua branch)
- Vai testar uma feature por dias antes de mergear
- Tem deploy automático ligado em `main` e tu não quer publicar a cada commit

7. PROFISSIONALIZAR PERFIL

Hoje teu perfil é "tem 14 repos, gráfico verde". Pra parecer profissional:

7.1 Profile README (o card de boas-vindas)

Existe um repo mágico: se tu cria um repo com **o mesmo nome do teu usuário** (`ImpulsoDigital063/ImpulsoDigital063`) e coloca um `README.md` dentro, ele aparece no topo do teu perfil.

Como criar:

1. `github.com/new`
2. Repository name: `ImpulsoDigital063` (exatamente igual ao user)
3. Public + "Add a README file"
4. Create repository
5. Edita o README.md

Template enxuto pra tu (copia e adapta):

Impulso Digital

Agência de tráfego, performance e tecnologia.

Construímos LPs, SaaS e automações pra negócios que querem crescer com método.

****O que faço aqui no GitHub:****

- Landing pages performáticas em Next.js + Tailwind
- SaaS multi-tenant em Supabase + Next
- Automações em Node e Python
- Integrações com Meta Ads, WhatsApp, Stripe

****Stack principal:**** Next.js 15 · TypeScript · Tailwind · Supabase · Vercel · Anthropic API

****Projetos em destaque:**** ver pinned abaixo.

impulsodigital.app.br · [Instagram @impulsodigital063](https://instagram.com)

7.2 Pinned repositories

Os 6 cards na home. **Hoje saíram automático** (Minha---Landing, impulso-digital-nextjs, locajv-landing, ev-suplementos, criativosdoceu, MPN-On).

Faz manual:

1. Vai pro perfil → "Customize your pins"
2. Escolhe 6 que **representam teu melhor trabalho** e mostram variedade
3. Critério: deploy ao vivo > README bom > stack moderna > recente

Sugestão pros teus 6 atuais:

- `impulso-digital-nextjs` — site institucional (vitrine própria)
- `ev-suplementos` — LP de cliente real com resultado
- `MPN-On` — projeto de mídia/automação
- 3 dos melhores LPs (escolha visual + técnica)

Não fixa: repos vazios, abandonados, ou que tu não conseguiria explicar numa entrevista.

7.3 Foto, bio, link

Settings → Profile:

- **Picture:** logo Impulso (não tua cara — é conta de empresa)
- **Name:** Impulso Digital
- **Bio:** "Agência de tráfego e tecnologia. SaaS, LPs, automação. 📍 Brasil"
- **URL:** impulsodigital.app.br
- **Twitter/X:** se tu posta
- **Email público:** contato@... (não o pessoal)

8. PROFISSIONALIZAR REPOS

Cada repo individual também precisa de capricho. Aplica este checklist por repo:

8.1 Nome bom

- ☒ `lp-clinica-fisio-rj` — claro, kebab-case
- ☐ `Minha---Landing` — confuso, hífen em excesso
- ☐ `teste`, `projeto1` — anônimo

Como renomear: Settings → General → Repository name → Rename.

8.2 Description + topics

No topo de cada repo, à direita do título, tem um ⚙️. Clica.

- **Description:** 1 linha. Ex: "Landing page de captação para clínica de fisioterapia no Rio. Next 15 + Tailwind + integração Meta."
- **Website:** URL do deploy (Vercel). Aparece como link clicável.

- **Topics:** tags. Adiciona: `nextjs`, `tailwindcss`, `typescript`, `landing-page`, `supabase`, etc. **Topics são indexados pelo GitHub** — outras pessoas acham teu repo buscando por `landing-page nextjs`.

8.3 README do projeto

TODO repo público precisa de README. Sem README, o repo parece morto.

Template enxuto (cola na raiz como README.md):

```
# Nome do Projeto

> 1 linha do que isso faz.

Captura screenshot e cola aqui: ![screenshot](docs/cover.png)

## Demo
[clinicafisio.app.br](https://clinicafisio.app.br)

## Stack
- Next.js 15 (App Router)
- TypeScript
- Tailwind CSS
- Supabase (auth + db)
- Vercel (deploy)

## Como rodar local
```bash
git clone https://github.com/ImpulsoDigital063/nome.git
cd nome
npm install
cp .env.example .env # preenche com tuas chaves
npm run dev # abre em localhost:3000
```

## Estrutura

```
src/
 app/ ← rotas Next (App Router)
 components/ ← componentes reutilizáveis
 lib/ ← helpers (supabase, utils)
 public/ ← imagens, fontes, favicons
```

## Variáveis de ambiente

Ver `.env.example`. Chaves obrigatórias:

- `NEXT_PUBLIC_SUPABASE_URL`
- `SUPABASE_SERVICE_ROLE_KEY` (server-side only)
- `RESEND_API_KEY` (envio de email)

## Licença

MIT (ou Proprietary se for de cliente)

### ### 8.4 Screenshots (pasta /docs)

Cria pasta `docs/` na raiz e joga:

- `cover.png` - screenshot da home, 1600x900
- `mobile.png` - versão mobile
- `dashboard.png` - se tiver área logada

Referência no README com `[alt](docs/cover.png)`.

**\*\*Por que importa:\*\*** quem entra no repo julga em 5 segundos pela imagem. Sem screenshot, ele clica fora.

### ### 8.5 Licença

Settings → General → "Add a license"

- **\*\*MIT\*\*** - qualquer um pode usar/modificar, só preserva créditos. Padrão pra projeto open.
- **\*\*Apache 2.0\*\*** - parecido com MIT mas com proteção de patente.
- **\*\*AGPL-3.0\*\*** - força quem usa a abrir o código também. Pesado.
- **\*\*Sem licença\*\*** = "todos direitos reservados" implícito. Outros não podem usar legalmente.

**\*\*Pra projeto de cliente pago:\*\*** sem licença ou "Proprietary" no README. Repo deve ser **\*\*privado\*\***.

**\*\*Pra portfólio open:\*\*** MIT.

### ### 8.6 Badges (opcional mas dá um look pro)

No topo do README:

```
```markdown
![Next.js](https://img.shields.io/badge/Next.js-15-black?logo=nextdotjs)
![TypeScript](https://img.shields.io/badge/TypeScript-5-blue?logo=typescript)
![Vercel](https://img.shields.io/badge/Deploy-Vercel-black?logo=vercel)
```

Gerador: shields.io.

9. COLABORAÇÃO

Cenário real: Lucas Passos vai mexer no teu GitHub. Como funciona sem tu dar a chave do cofre?

9.1 Os 3 níveis de acesso

Nível	O que pode	Quando dar
Nenhum (público)	só ler/clonar/forkar	sempre que repo é público
Collaborator	pode commitar direto na main	só pra parceiros de confiança em projetos compartilhados
Admin	tudo, incluindo deletar o repo	NUNCA pra terceiros

Pra colaboração pontual (caso Lucas) = ele NÃO precisa de acesso de collaborator. Ele forka, mexe no fork dele, abre Pull Request. Tu revisa e merge.

9.2 Fluxo Fork + PR (passo a passo)

Do lado do Lucas:

1. Abre github.com/ImpulsoDigital063/impulso-digital-nextjs
2. Clica "Fork" (canto superior direito) → cria cópia em github.com/lspassos1/impulso-digital-nextjs
3. Clona o fork local: `git clone https://github.com/lspassos1/impulso-digital-nextjs.git`
4. Cria branch: `git checkout -b feat/profile-readme`
5. Edita, commita, faz push
6. No GitHub, vai pro fork dele → clica "Compare & pull request" → escreve descrição → "Create pull request"

Do teu lado:

1. Recebe notificação no email/GitHub
2. Abre o PR — vê:
 - **Files changed** → diff visual (verde = adicionado, vermelho = removido)
 - **Conversation** → comentários
 - **Commits** → lista de commits do PR
3. Tu pode:

- Comentar linha por linha
- Pedir mudanças (Request changes)
- Aprovar (Approve)
- **Mergear** (Merge pull request)

Se for bom: clica "Merge pull request" → "Confirm merge". As mudanças do Lucas viram parte da tua `main`.

9.3 Adicionar collaborator (só se for inevitável)

Settings → Collaborators → Add people → busca por username (`lspassos1`) → escolhe role (Write é o padrão pra colaborar).

Se fizer isso: ativa Branch Protection na `main` (próxima seção). Senão ele commita direto sem revisão.

9.4 Branch Protection (proteger a main)

Settings → Branches → Add rule → Branch name pattern: `main`

Marca:

- ☒ Require a pull request before merging
- ☒ Require approvals (1)
- ☒ Require status checks to pass (se tiver CI)
- ☒ Do not allow bypassing the above settings

Resultado: nem tu nem o Lucas conseguem commitar direto em `main`. Tudo passa por PR. Trava bobeira.

9.5 Code review — o que olhar num PR

- A mudança faz o que a descrição do PR diz?
- Quebrou algo que já funcionava?
- Tem `.env` ou senha vazada nos arquivos modificados?
- Mensagens de commit fazem sentido?
- Mexeu em arquivo crítico (next.config, supabase, package.json) sem necessidade?

Se algo te incomoda: comenta na linha, marca "Request changes". Ele corrige, atualiza o PR (sem fechar — o mesmo PR ganha commits novos), tu re-revisa.

10. SEGURANÇA

10.1 Coisas que NUNCA podem ir pro Git

- `.env`, `.env.local`, `.env.production` (têm SENHAS)
- Arquivos `*.pem`, `*.key`, `*.p12`
- Tokens hardcoded em código: `const KEY = "sk-abc123 ..."` ← NUNCA
- `service-account.json` (Google), `firebase-admin.json`
- Backups de banco (`.sql`, `.dump`)
- Senhas no README ou docs/

Se vazou: Git **não esquece**. Deletar o arquivo e dar push NÃO resolve — o histórico ainda tem. Soluções:

1. **Revoga a chave imediatamente** na origem (Supabase, OpenAI, etc.) — essa é a única coisa que importa
2. Opcional: reescrever histórico com `git filter-repo` ou BFG Repo-Cleaner (complicado)

10.2 GitHub Secrets (jeito certo de usar chaves em Actions)

Settings → Secrets and variables → Actions → New repository secret.

Cria `SUPABASE_SERVICE_KEY` aqui. No workflow do GitHub Actions referencia como `${{ secrets.SUPABASE_SERVICE_KEY }}`. Nunca aparece em log.

10.3 2FA na conta

Settings (da conta, não do repo) → Password and authentication → Two-factor authentication.

OBRIGATÓRIO. Sem 2FA, basta vazar tua senha e o atacante apaga tudo. Usa app autenticador (Authy, Google Authenticator, 1Password).

10.4 SSH keys (alternativa a senha pra push)

Hoje tu provavelmente loga com usuário+senha (ou token) toda vez que dá push. Configura SSH:

```
ssh-keygen -t ed25519 -C "edubchaves5@gmail.com"
# enter, enter (sem passphrase pra simplicidade) ou define passphrase

# copia a chave pública
cat ~/.ssh/id_ed25519.pub
```

GitHub → Settings → SSH and GPG keys → New SSH key → cola.

Depois usa URL SSH em vez de HTTPS:

```
git remote set-url origin git@github.com:ImpulsoDigital063/projeto.git
```

Pronto. `git push` direto sem senha.

11. ERROS COMUNS

"rejected — non-fast-forward"

Tu commitou local, mas alguém (ou tu de outra máquina) commitou no remote antes. Resolve:

```
git pull --rebase
# resolve conflitos se aparecer
git push
```

Merge conflict

Dois commits mexeram nas mesmas linhas. Git pede tua ajuda. Abre o arquivo conflitado:

```
<<<<<< HEAD
texto que TU tem local
=====
texto que VEIO do remote
>>>>>> origin/main
```

Edita manualmente, escolhe qual versão (ou junta), apaga as marcações `<<<`, `===`, `>>>`. Depois:

```
git add arquivo-resolvido.tsx
git commit -m "resolve conflito"
git push
```

Comitei errado, quero desfazer

Se ainda **NÃO** pushou:

```
git reset --soft HEAD~1      # desfaz último commit, mantém arquivos modificados
git reset --hard HEAD~1      # APAGA último commit E mudanças. CUIDADO.
```

Se já pushou:

```
git revert HEAD              # cria commit novo que desfaz o anterior. Seguro.
git push
```

NUNCA use `git push --force` em branch compartilhada. Apaga commits dos outros.

Deletei algo sem querer

```
git reflog                  # mostra TODOS os movimentos do HEAD (mesmo "deletados")
git checkout <hash>         # volta pra antes
```

Git praticamente não apaga nada de verdade nos primeiros 30 dias. Calma.

Pushei pro repo errado

Verifica:

```
git remote -v
```

Se tá errado:

```
git remote set-url origin https://github.com/ImpulsoDigital063/certo.git
git push
```

Esqueci de `.gitignore` e comitei `node_modules`

```
git rm -r --cached node_modules
echo "node_modules/" >> .gitignore
git add .gitignore
git commit -m "chore: gitignore node_modules"
git push
```

Mensagem do último commit tá errada

```
git commit --amend -m "mensagem certa"
# se já pushou:
git push --force-with-lease    # mais seguro que --force
```

12. CHEATSHEET

Imprime ou cola no Notion. Os comandos que tu vai usar 90% do tempo:

# começar	
git init	# iniciar versionamento
git clone <url>	# clonar repo existente
# ver estado	
git status	# o que mudou
git log --oneline -10	# últimos 10 commits
git diff	# ver mudanças não-staged
git diff --staged	# ver mudanças staged
# trabalhar	
git add .	# marca todos os arquivos
git add arquivo.tsx	# marca um
git commit -m "feat: nova feature"	# commita
git push	# envia
git pull	# recebe
# branches	
git branch	# lista
git checkout -b feature/x	# cria + troca
git checkout main	# troca
git merge feature/x	# mescla (estando em main)
git branch -d feature/x	# deleta local
# remote	
git remote -v	# lista remotes
git remote add origin <url>	# adiciona
git remote set-url origin <url>	# muda url
# desfazer	
git reset --soft HEAD~1	# desfaz commit, mantém mudanças
git revert HEAD	# desfaz commit criando novo
git restore arquivo.tsx	# descarta mudanças do arquivo
git stash	# guarda mudanças temporariamente
git stash pop	# recupera
# inspecionar	
git log --all --graph --oneline	# árvore visual
git blame arquivo.tsx	# quem mexeu em cada linha
git show <hash>	# detalhes de um commit
git reflog	# histórico de movimentos do HEAD

13. GLOSSÁRIO PT-EN

Inglês (GitHub)	Português	O que é
Repository / Repo	Repositório	Projeto versionado
Commit	Commit	"Foto" do projeto num momento
Branch	Branch / Ramo	Linha do tempo paralela
Fork	Fork / Bifurcação	Cópia do repo pro teu GitHub
Pull Request (PR)	Pedido de Merge	Proposta de mudança vinda de fork ou branch
Merge	Merge / Mesclagem	Juntar duas branches
Conflict	Conflito	Quando duas mudanças batem na mesma linha
Push	Push / Enviar	Mandar commits locais pro remote
Pull	Pull / Puxar	Trazer commits do remote pro local
Clone	Clonar	Baixar repo completo (com histórico)
Stage	Marcar (preparar pra commit)	<code>git add</code>
Stash	Engavetar	Guardar mudanças sem commitar
Issue	Issue / Tarefa	Bug ou pedido de feature
Release	Release / Lançamento	Versão publicada com notas
Tag	Tag / Marcador	Etiqueta num commit (ex: v1.0.0)
Star	Estrela	Curtir/favoritar repo
Watch	Observar	Receber notificações do repo
Following	Seguir	Acompanhar outro user
Achievement	Conquista	Medalhinhas (Pull Shark, etc.)
Pinned	Fixado	Repos destacados no perfil
Owner	Dono	Usuário ou organização dona
Collaborator	Colaborador	Quem tem write access
Action / Workflow	Action / Fluxo	Automação (CI/CD)
Secret	Secret / Segredo	Variável protegida (senha, token)

14. CHECKLIST — "DEIXAR PROFISSIONAL EM 3 HORAS"

Marca conforme faz. Ordem importa.

Hora 1 — Conta + perfil

- ☐ Settings → 2FA ativado
- ☐ Settings → Profile → foto da Impulso
- ☐ Settings → Profile → bio: "Agência de tráfego e tecnologia. SaaS, LPs, automação."
- ☐ Settings → Profile → website: `impulsodigital.app.br`
- ☐ Settings → Profile → email público: `contato@impulsodigital.app.br`
- ☐ Criar repo `ImpulsoDigital063/ImpulsoDigital063` com README (template em [§7.1](#))
- ☐ SSH key configurada (opcional mas vale, [§10.4](#))

Hora 2 — Curadoria de repos

- ☐ Repositories → lista os 14 → identifica:
 - ☐ **Manter público** (portfólio bom) — vai limpar
 - ☐ **Tornar privado** (cliente pago, sensível) — Settings → Danger Zone → Change visibility
 - ☐ **Arquivar** (morto mas não dá pra deletar) — Settings → Archive
 - ☐ **Deletar** (lixo de teste) — Settings → Danger Zone → Delete
- ☐ Pinned: customizar com os 6 melhores (Customize your pins)
- ☐ Pros 6 fixados: cada um precisa de description + topics + website (deploy)

Hora 3 — README dos 3 principais

Pega os 3 repos top (não os 6 — começa pequeno):

- ☐ Repo 1: README com template de [§8.3](#)
- ☐ Repo 1: screenshot em `docs/cover.png` + linkar no README
- ☐ Repo 2: idem
- ☐ Repo 3: idem
- ☐ Pra cada um: license (MIT se público) + `.gitignore` correto + `.env.example` sem segredo

Resultado depois das 3 horas: teu perfil vira "vitrine" em vez de "depósito". Tu pode mandar a URL pra qualquer cliente, dev parceiro, ou candidato a freelancer.

ANEXOS

A. Comandos pra rodar AGORA (validar setup)

```
git --version           # confirma git instalado
git config --global user.name # confirma nome
git config --global user.email # confirma email
ssh -T git@github.com    # testa SSH (deve dizer "Hi ImpulsoDigital063")
```

Se algum sair errado:

```
git config --global user.name "Eduardo Barros"
git config --global user.email "edubchaves5@gmail.com"
```

B. Pra estudar depois (quando quiser ir mais fundo)

- Pro Git book** — git-scm.com/book/pt-br/v2 — grátis, oficial, em português
- GitHub Skills** — skills.github.com — tutoriais interativos no próprio GitHub
- Conventional Commits** — conventionalcommits.org — padrão de mensagens
- Shields.io** — shields.io — gerador de badges

C. Quando chamar o Lucas

Depois que tu fez a Hora 1 e Hora 2 do checklist sozinho, **aí** manda o GitHub pra ele. Por dois motivos:

- Tu vai entender o que ele tá fazendo (não vira mágica de programador)
- O escopo do trabalho dele encolhe — sobra só "polir os READMEs dos top 6" e "estruturar o Profile README", que dá pra fazer em ~3h dele

Sem isso, tu paga ele (ou queima permuta) pra fazer trabalho de 10min que tu mesmo faz.

Versão: 1.0 · 24/05/2026 **Autor:** Claude (Verbo/Code) · revisão de Eduardo Barros **Próxima revisão:** quando tu tiver feito o checklist e identificar lacunas